Don't remove the default PageController route in your project—normally you would delete this route, but keep it so you don't get test failures in Chapter 10, Testing Elixir, on page 171.

We'll work through each of these four routes by the end of the chapter.

We could have used resources/4 for our routes, but it doesn't quite fit with the URL structure. So we use the longer form instead.

If you tried to start the app now, you would see many warnings about MessageController being undefined. Let's create that module and add our first action:

phone_app/lib/phone_app_web/controllers/message_controller.ex

```
Line 1  defmodule PhoneAppWeb.MessageController do
  -       use PhoneAppWeb, :controller
  -
  -       plug :load_conversation_list
  5
  -       def index(conn, _params) do
  -         case conn.assigns.conversation_list do
  -           [%{contact: contact} | _] ->
  -             path = ~p(/messages/#{contact.id})
  10            redirect(conn, to: path)
  -
  -           [] ->
  -             redirect(conn, to: ~p(/messages/new))
  -         end
  15      end
  -
  -       defp load_conversation_list(conn, _params) do
  -         conversations = PhoneApp.Conversations.load_conversation_list()
  -         assign(conn, :conversation_list, conversations)
  20      end
  - end
```

A lot is going on here, so let's break it all down. The first thing you'll notice is the use function on line 2. This is the same pattern that was used in the router earlier—it sets up the necessary code to make our controller work.

The plug on line 4 acts like a before_action filter in Rails. The load_conversation_list/2 function will be called for every action in this controller. Each page of this app has a sidebar that contains all conversations, so the plug ensures the data is always loaded.

The load_conversation_list/2 function calls the Conversations context module to load the conversation list. (You added these functions in the previous chapter.) assign/3 is used on line 19 to set this data in a special location called assigns. Assigns can be accessed in the conn struct—as on line 7—and in templates.